Scraper Conversations

> From: https://gemini.google.com/app/5f5ac19552fac264

you asked

message time: 2026-01-20 23:49:03

So basically, I want to scrape the official lebanese gazette, and then turn it into a functional tool for democracy, enhanced search etc, for the scraping, I want to go year by year, and download all the issues in a year, I will want to format the titles obviously - in some cases, the gazette is a full pdf, in some other cases, its just individual pages which need to be made into a full pdf and formatted correctly with the date etc

---

gemini response

That is an ambitious and highly impactful project. Since you are looking to build a "functional tool for democracy," the quality of the initial data ingest (the scraping and PDF reconstruction) is the most critical foundation for the search and analysis layers that follow.

The Lebanese Official Gazette (Al-Jarida Al-Rasmiya) website—often hosted via the **JOANNA** system or the **Presidential/PCM** portals—is notorious for inconsistent formatting.

Here is a technical strategy for the scraper and the PDF reconstruction engine.

1\. The Scraping Strategy: "Year-by-Year"

Since the URL structures for the Gazette often change based on the decade, you should build a **Mapping Layer** first.

- **Logic:** Don't just scrape; create a local manifest (JSON or CSV) that records the Year, Issue Number, and the "Type" (Single PDF vs. Image-based pages).
- **Tools:** Use `Selenium` or `Playwright` if the site uses heavy JavaScript (common for the page-flipping viewers). If it's a simple directory, `BeautifulSoup` with

`requests` is faster.

- **Architecture:**
  - `scripts/fetch_manifest.py`: Navigates the calendar/archive and collects all direct links.
  - `scripts/download_raw.py`: Downloads files into a structured directory like `raw_data/1998/issue_24/`.

2\. Handling the Two PDF Scenarios

You will need a pipeline that detects the file type and routes it accordingly.

Scenario A: The full PDF exists

If the download is a single PDF, your main task is **renaming and metadata attachment**.

- **Tool:** `PyMuPDF` (fitz) is excellent for this.
- **Action:** Extract the date from the first page text (using regex for Arabic dates) to ensure the filename matches the actual content, e.g., `1998-05-14-Issue-20.pdf`.

Scenario B: Individual Pages (The "Stitcher")

When the Gazette is served as separate image pages or 1-page PDFs, you need to reconstruct the "Issue" as a single searchable document.

Step	Task	Recommended Tool
1	**Download All**	Loop through page IDs until a 404 is returned or the "Next" button disappears.
2	**Sort**	Ensure pages are ordered numerically (watch out for `page_1.jpg` vs `page_10.jpg`—use natural sorting).
3	**Convert/Merge**	Use `img2pdf` for images (lossless) or `PyMuPDF` for PDF merging.
4	**OCR**	**Crucial for search.** Since these are often scans, use `ocrmypdf` with the `ara` (Arabic) language pack.

3\. Proposed Folder Structure

To keep this organized for your "enhanced search" tool later, I recommend this structure:

```
/lebanese-gazette-data
  /raw
    /1998
      /issue_20 (contains individual .jpg or .pdf pages)
  /processed
    /1998
      1998-05-14_Issue-20_Searchable.pdf
  /metadata
    gazette_index.json  (links issue # to date and file path)
```

4\. Code Snippet: The Stitcher (Scenario B)

If you have a folder of images for one issue, this Python logic will create the PDF:

```
import img2pdf
import os

def stitch_issue(input_folder, output_path):
    # Natural sort to ensure page 2 comes before page 10
    images = sorted([os.path.join(input_folder, f) for f in o
s.listdir(input_folder) if f.endswith('.jpg')],
                    key=lambda x: int(''.join(filter(str.isdi
git, x))))

    with open(output_path, "wb") as f:
        f.write(img2pdf.convert(images))
```

How can I best help you next?

Would you like me to:

1. **Write a Selenium script** specifically for one of the known Gazette portals (e.g., the PCM or Joanna site)?
2. **Help with the Arabic OCR configuration** to make sure the "enhanced search" can actually read the text?

3. **Design the database schema** for the "functional tool" so you can search by Decree Number or Law Category?

> From: https://gemini.google.com/app/df4c843396521bb8

you asked

message time: 2026-01-26 13:31:19

I want to build a scraper to automatically download the gazette issues from the official lebanese gazette and have it be downloaded with teh correct dates

---

gemini response

Building a scraper for the Official Lebanese Gazette is a great way to build a searchable archive. The gazette is typically hosted on the **Presidency of the Council of Ministers (PCM)** website.

Scraping Strategy

The main challenge with the Lebanese Gazette website (`pcm.gov.lb`) is that it often uses a structured navigation menu for years and months rather than a simple flat list. To get the correct dates, you'll need to extract the date text associated with each download link.

1\. Identify the Target URLs

The archive is generally organized by year. For 2026, the landing page is: `http://www.pcm.gov.lb/arabic/landing.aspx?pageid=9` (for the main gazette section).

2\. Required Tools

You will need a few Python libraries:

- `requests`: To fetch the page content and download the PDFs.
- `BeautifulSoup` **(bs4)**: To parse the HTML and find the download links.

- `re`: To extract dates from the text using regular expressions.

Sample Python Scraper

Here is a foundational script to get you started. It targets the year 2025/2026 landing pages and looks for PDF links.

```
import os
import requests
from bs4 import BeautifulSoup
import re

# Base URL for the Gazette archive
BASE_URL = "<http://www.pcm.gov.lb/arabic/subpgoldjo.aspx?pag
eid=3836>"

def download_gazette():
    response = requests.get(BASE_URL)
    if response.status_code != 200:
        print("Failed to access the website.")
        return

    soup = BeautifulSoup(response.content, 'html.parser')

    # Look for all anchor tags that link to .pdf files
    links = soup.find_all('a', href=re.compile(r'.pdf$'))

    if not os.path.exists('lebanese_gazette'):
        os.makedirs('lebanese_gazette')

    for link in links:
        pdf_url = link.get('href')
        # Ensure the URL is absolute
        if not pdf_url.startswith('http'):
            pdf_url = "<http://www.pcm.gov.lb>" + pdf_url
```

```
        # Get the text associated with the link (often contai
ns the date/issue number)
        link_text = link.get_text(strip=True)

        # Sanitize filename (remove Arabic characters or keep
them as per preference)
        # Often the date is in the format YYYY-MM-DD or DD-MM
-YYYY in the text
        filename = f"{link_text.replace('/', '-')}.pdf"

        print(f"Downloading: {filename}...")

        pdf_data = requests.get(pdf_url).content
        with open(f"lebanese_gazette/{filename}", 'wb') as f:
            f.write(pdf_data)

if __name__ == "__main__":
    download_gazette()
```

Important Considerations

- **Arabic Text Handling**: The link text will be in Arabic (e.g., "- ٤٢ العدد ١٥/١٠/٢٠٢٥"). If you want the filenames in English, you'll need a mapping for months or a regex to extract the digits for a format like `2025-10-15.pdf`.
- **Dynamic Content**: If the website uses JavaScript to load the lists (common in newer government portals), you might need **Selenium** or **Playwright** instead of `requests`.
- **Rate Limiting**: Be polite\! Use `time.sleep(2)` between downloads so the server doesn't block your IP.
- **Archive Structure**: The PCM site often breaks the archive down by year into different `pageid` parameters. You may need to create a list of IDs to crawl multiple years.

Would you like me to refine the script to specifically extract and format the Arabic dates into a standard `YYYY-MM-DD` filename?

you asked

message time: 2026-01-26 13:37:53

https://jo.pcm.gov.lb/allimages.php?issueId=9236

---

gemini response

The URL provided (`jo.pcm.gov.lb/allimages.php?issueId=9236`) indicates that this version of the Lebanese Gazette portal uses a database-driven system where each issue is identified by a unique `issueId`.

To build a scraper that downloads these with the **correct dates**, you need a two-step process:

1. **Metadata Extraction:** Scrape the main archive page to map the `issueId` to its corresponding date.
2. **Download:** Navigate to the specific issue page (or the direct PDF link) to save the file.

Recommended Scraping Logic (Python)

Because this site often uses specific table structures for its archive, you can use the following approach to extract the date and the PDF.

```
import requests
from bs4 import BeautifulSoup
import os
import re

# Base URL for the archive list (where the dates and issueIds
are listed)
ARCHIVE_URL = "<https://jo.pcm.gov.lb/archive.php>" # You may
need to adjust this to the actual list page
```

```
BASE_PDF_URL = "<https://jo.pcm.gov.lb/>"

def scrape_gazette():
    # 1. Fetch the page containing the list of issues
    response = requests.get(ARCHIVE_URL)
    soup = BeautifulSoup(response.content, 'html.parser')

    # Create directory
    if not os.path.exists('lebanese_gazette'):
        os.makedirs('lebanese_gazette')

    # 2. Find the table rows containing Issue IDs and Dates
    # Note: You'll need to inspect the 'archive.php' page to
find the exact table class/id
    for row in soup.find_all('tr'):
        cells = row.find_all('td')
        if len(cells) >= 2:
            # Look for the date text (e.g., 15/01/2026)
            date_match = re.search(r'(\\d{2}/\\d{2}/\\d{4})',
cells[0].text)

            # Look for the link containing the issueId
            link = row.find('a', href=re.compile(r'issueId
='))

            if date_match and link:
                date_str = date_match.group(1).replace('/',
'-')
                issue_id = link.get('href').split('issueId=')
[-1]

                # Construct a clean filename: YYYY-MM-DD_Issu
eID.pdf
                # Reversing date for better chronological sor
ting (YYYY-MM-DD)
                date_parts = date_str.split('-')
```

```
                formatted_date = f"{date_parts[2]}-{date_part
s[1]}-{date_parts[0]}"
                filename = f"{formatted_date}_Issue_{issue_i
d}.pdf"

                # 3. Determine PDF URL
                # If the PDF link is direct:
                pdf_download_url = f"<https://jo.pcm.gov.lb/d
ownload_pdf.php?issueId={issue_id}>"

                print(f"Downloading {filename}...")
                download_file(pdf_download_url, filename)

def download_file(url, filename):
    try:
        r = requests.get(url, stream=True)
        if r.status_code == 200:
            with open(f"lebanese_gazette/{filename}", 'wb') a
s f:
                for chunk in r.iter_content(chunk_size=8192):
                    f.write(chunk)
    except Exception as e:
        print(f"Error downloading {filename}: {e}")

if __name__ == "__main__":
    scrape_gazette()
```

Key Technical Details for `jo.pcm.gov.lb`:

1. **Direct PDF vs. Image Viewer:** The URL you provided (`allimages.php`) suggests a gallery view. Look for a link on that page that says "Download PDF" or "تحميل النسخة الإلكترونية". Use the Network tab in your browser's Developer Tools (F12) to see the actual URL triggered when you click download. It is often `download.php?id=...` or similar.

2. **Date Formatting:** If the dates are in Arabic numerals (٣ ,٢ ,١), you will need a small helper function to convert them to standard integers:

```
def convert_arabic_numbers(text):
    arabic_digits = '٠١٢٣٤٥٦٧٨٩'
    english_digits = '0123456789'
    translation_table = str.maketrans(arabic_digits, engli
sh_digits)
    return text.translate(translation_table)
```

3. **Iteration:** Since you have the `issueId`, you could technically loop through IDs (e.g., from 9000 to 9236), but you wouldn't know the dates without visiting the page first. It is better to scrape the **Archive/Search results page** where the `issueId` and `Date` are listed side-by-side.

To help you further: If you can provide the HTML snippet of the page where the list of all issues appears, I can give you the exact `BeautifulSoup` selectors to use.

you asked

message time: 2026-02-01 23:42:48

give me a paragraph summary of what the lebanese official gazette is, how it works, etc etc

---

gemini response

The **Lebanese Official Gazette** (Arabic: *Al-Jarīdah Al-Rasmīyah*) is the central state organ used for the formal publication of all legal and administrative instruments in Lebanon. Established in **1922** during the French Mandate, it serves as the ultimate source of legal authority; under Lebanese law, most statutes, legislative decrees, and ministerial decisions only become legally binding and enforceable against the public after they have been published in the Gazette. It is typically published **weekly** and contains a comprehensive record of the state's activities, including new laws passed by Parliament, executive decrees from the

President or Council of Ministers, official appointments, government tenders, and announcements related to the commercial register and trademarks.

How it Works

- **Legal Force:** Publication in the Gazette creates a legal presumption that everyone is aware of the law (the principle of "ignorance of the law is no excuse"). Most laws specify a waiting period—often **30 days**—between publication and coming into force, unless an emergency status is granted.
- **Structure:** The Gazette is organized into sections: the first usually contains primary legislation and decrees, while subsequent sections cover administrative decisions and public notices.
- **Language:** It is issued exclusively in **Arabic**, which is the official language of the Lebanese administration.
- **Archives:** While historically a physical print publication, recent efforts have focused on digitizing the archives to improve transparency and public access, though many older records remain difficult to search without specialized databases.

Would you like me to find more specific details on how to access their recent digital archives for your project?

> From: https://gemini.google.com/app/ed096ed06689a2c0

you asked

message time: 2026-02-02 12:17:48

help me build a scraper for the official lebanese gazette to download all the issues in a year

---

gemini response

Building a scraper for the Official Lebanese Gazette (الجريدة الرسمية) is a great way to centralize access to these documents. The official portal is hosted by the **Presidency of the Council of Ministers (PCM)**.

The primary challenge is that the site uses a mix of static listings and ASPX-based navigation. Below is a structured approach to building your scraper using Python.

1\. Target URL Structure

The official landing page for the gazette issues is:
`https://jo.pcm.gov.lb/` or the older archive structure at `http://www.pcm.gov.lb/`.

Typically, the issues are organized by **Year** and **Issue Number**. A direct URL pattern often looks like this:
`https://jo.pcm.gov.lb/DownloadIssue?issueId={ID}` or links found within the page results.

2\. Recommended Tech Stack

- **Python**: The best language for this task due to its library ecosystem.
- **Requests**: For downloading the PDF files.
- **BeautifulSoup4**: For parsing the HTML to find the download links.
- **Selenium or Playwright**: Necessary if the page requires interaction (like selecting a year from a dropdown) or if the links are generated via JavaScript.

3\. Scraper Logic Breakdown

Phase A: Setup and Year Selection

Since you want to download by year, your script needs to:

1. Navigate to the search/archive page.
2. Select the desired year from the dropdown.
3. Trigger the search button (this often uses an ASP.NET `__doPostBack` function).

Phase B: Extraction (The Python Code)

Here is a simplified template using `requests` and `BeautifulSoup`.

> **Note:** If the site uses heavy JavaScript, you will need to replace `requests` with `Selenium`.

```
import os
import requests
from bs4 import BeautifulSoup

def download_gazette_year(year):
    base_url = "<https://jo.pcm.gov.lb/>" # Update based on c
urrent active URL
    search_url = f"{base_url}/search-endpoint-or-page" # Targ
et the specific year page

    # Create directory for the year
    if not os.path.exists(str(year)):
        os.makedirs(str(year))

    # 1. Fetch the page content
    response = requests.get(search_url)
    soup = BeautifulSoup(response.text, 'html.parser')

    # 2. Find all PDF links (this selector depends on the sit
e's HTML structure)
    # Usually, it's an <a> tag with 'pdf' in the href or a sp
ecific class
    links = soup.find_all('a', href=True)

    for link in links:
        href = link['href']
        if ".pdf" in href.lower():
            file_url = base_url + href if not href.startswith
('http') else href
            file_name = href.split('/')[-1]

            print(f"Downloading {file_name}...")

            # 3. Download the file
            pdf_data = requests.get(file_url).content
```

```
            with open(f"{year}/{file_name}", 'wb') as f:
                f.write(pdf_data)

# Example usage
# download_gazette_year(2024)
```

4\. Key Challenges to Watch For

- **Dynamic Loading**: If the list of issues doesn't appear in the initial HTML source (right-click -\> View Page Source), the site is dynamic. You will need to use **Selenium** to "click" the year and wait for the table to load.
- **Rate Limiting**: Government servers can be sensitive. Add `time.sleep(2)` between downloads to avoid being blocked.
- **Naming Conventions**: Official names are often in Arabic. You may want to rename files based on the issue number and date found in the table row rather than the cryptic PDF filename.

5\. Ethical & Legal Note

- **Robots.txt**: Check `https://jo.pcm.gov.lb/robots.txt` to see if they allow scraping.
- **Usage**: Since this is for a project on the Lebanese Gazette, ensure you are following "fair use" and not overwhelming their servers with high-concurrency requests.

Would you like me to help you write a more specific Selenium script that handles the dropdown selection for the year?

you asked

message time: 2026-02-02 12:24:39

https://jo.pcm.gov.lb/allimages.php?issueId=9236 this is the last issue of 2025

gemini response

To build a scraper for the Lebanese Gazette based on the URL structure you provided, you will need to handle a specific challenge: the site often displays issues as individual page images rather than a single PDF.

Based on the URL `https://jo.pcm.gov.lb/allimages.php?issueId=9236`, the site likely uses an incremental `issueId` system. To download all issues for a year, you can follow this logic:

1\. Reverse Engineering the URL

The URL structure suggests that `issueId` is the primary key.

- **Issue ID 9236** corresponds to the end of 2025.
- To find the start of the year, you would decrement the ID until you reach the first issue dated in January 2025.

2\. The Python Scraper Logic

You can use `Requests` and `BeautifulSoup` to find all image tags within that page and save them.

```
import os
import requests
from bs4 import BeautifulSoup
import time

def download_issue(issue_id):
    url = f"<https://jo.pcm.gov.lb/allimages.php?issueId={iss
ue_id}>"
    headers = {'User-Agent': 'Mozilla/5.0'}

    try:
        response = requests.get(url, headers=headers)
        soup = BeautifulSoup(response.text, 'html.parser')

        # Find all images in the issue
```

```
        # The site typically uses <img> tags for each page of
the gazette
        images = soup.find_all('img')

        if not images:
            print(f"No images found for Issue {issue_id}")
            return

        # Create a folder for the specific issue
        folder_path = f"Gazette_Issue_{issue_id}"
        os.makedirs(folder_path, exist_ok=True)

        for i, img in enumerate(images):
            img_url = img.get('src')
            if not img_url:
                continue

            # Ensure the image URL is absolute
            if not img_url.startswith('http'):
                img_url = "<https://jo.pcm.gov.lb/>" + img_ur
l

            img_data = requests.get(img_url).content
            with open(f"{folder_path}/page_{i+1}.jpg", 'wb')
as f:
                f.write(img_data)

        print(f"Successfully downloaded Issue {issue_id}")

    except Exception as e:
        print(f"Error downloading {issue_id}: {e}")

# Example: Loop through a range of IDs for the year
# You will need to determine the starting ID for 2025 by chec
king the site
start_id = 9100  # Placeholder: check the site for the first
```

```
Jan 2025 ID
end_id = 9236

for current_id in range(start_id, end_id + 1):
    download_issue(current_id)
    time.sleep(1) # Be respectful to the server
```

3\. Key Considerations for your Project

- **Image to PDF**: Since the `allimages.php` page serves JPGs, you might want to add a post-processing step using the `img2pdf` library to combine the downloaded pages into a single PDF per issue.
- **Missing IDs**: The `issueId` sequence might have gaps. Your script should be robust enough to skip an ID if the page returns a 404 or "No images found."
- **Metadata**: The `allimages.php` page usually contains the Date and Issue Number in the text. You can scrape this text using `soup.find()` to rename your folders from `Issue_9236` to something more descriptive like `Issue_52_26-12-2025`.

4\. Refining the "Year" Search

If you don't want to guess the `start_id`, you can scrape the main archive page (`https://jo.pcm.gov.lb/jsearch.php`) first. You can filter by year there, extract all the `issueId` links, and then pass that list to the downloader script above.

you asked

message time: 2026-02-02 12:26:39

when i click print for example https://jo.pcm.gov.lb/pdfs/printall-1.php?issueId=9156&current=1

---

gemini response

When you click "print" on the site, it generates a URL like `https://jo.pcm.gov.lb/pdfs/printall-1.php?issueId=9156&current=1`. This is a critical discovery for your scraper because this endpoint returns a **compiled PDF file** rather than a webpage full of individual images.

By using this "printall" URL structure, you can bypass the need to download multiple JPGs and merge them yourself.

How to use this for your Scraper:

1. **Direct PDF Access**: Instead of scraping `allimages.php?issueId=XXXX`, point your scraper directly to `pdfs/printall-1.php?issueId=XXXX`.
2. **Simplified Download Code**: You can now download the entire issue in one request.

```
import requests

def download_full_pdf(issue_id):
    # The 'current=1' parameter tells the server to genera
te the full issue
    pdf_url = f"<https://jo.pcm.gov.lb/pdfs/printall-1.ph
p?issueId={issue_id}&current=1>"
    response = requests.get(pdf_url, stream=True)

    if response.status_code == 200:
        with open(f"Lebanese_Gazette_{issue_id}.pdf", 'w
b') as f:
            f.write(response.content)
        print(f"Issue {issue_id} downloaded successfull
y.")
    else:
        print(f"Failed to download Issue {issue_id}. Statu
s: {response.status_code}")
```

3. **PDF Identification**: The file content confirms that this endpoint produces a standard `%PDF-1.4` document. It includes embedded fonts like `XBRiyaz` and

`DejaVuSerifCondensed-Bold` to correctly render the Arabic text of the Gazette.

4. **Metadata Extraction**: While the PDF itself is a binary file , the server-side generation date is embedded in the file's metadata (e.g., `20260202112642`). You can use this to track when your downloads were generated.

Pro-Tip for your Project:

The `issueId` is the key. Since you know `9236` is the last issue of 2025, you can simply run a loop from your starting ID up to `9236` using this PDF link format to automate the collection of the entire year's archive.

you asked

message time: 2026-02-02 12:40:46

okay and if i know the issue range can we also add that and a ticker to avoid suspission?

---

gemini response

To build a robust and respectful scraper for the Lebanese Gazette project, we can combine the direct PDF download link we discovered with a loop that includes a "ticker" (randomized delay). This approach mimics human browsing behavior and prevents the server from being overwhelmed by too many requests in a short window.

1\. Scraper Structure with Ticker

This script uses the `time` and `random` libraries to create a variable "sleep" period between downloads.

```
import os
import time
import random
import requests
```

```python
def download_gazette_archive(start_id, end_id, output_dir="Lebanese_Gazette_2025"):
    # Create the folder if it doesn't exist
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
        print(f"Created directory: {output_dir}")

    # The PDF endpoint discovered from the print link
    base_url = "<https://jo.pcm.gov.lb/pdfs/printall-1.php>"

    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36',
        'Referer': '<https://jo.pcm.gov.lb/>'
    }

    for issue_id in range(start_id, end_id + 1):
        print(f"--- Processing Issue ID: {issue_id} ---")

        # Construct the full URL
        params = {
            'issueId': issue_id,
            'current': 1
        }

        try:
            # Stream the download to handle large PDFs efficiently
            response = requests.get(base_url, params=params, headers=headers, stream=True, timeout=30)

            if response.status_code == 200:
                file_path = os.path.join(output_dir, f"Issue_{issue_id}.pdf")
                with open(file_path, 'wb') as f:
```

```
                    for chunk in response.iter_content(chunk_
size=8192):
                        f.write(chunk)
                print(f"Successfully saved Issue {issue_i
d}.")
            else:
                print(f"Skipping Issue {issue_id}: Received S
tatus {response.status_code}")

        except Exception as e:
            print(f"Error occurred at ID {issue_id}: {e}")

        # --- The Ticker (Anti-Suspicion) ---
        # Pick a random delay between 5 and 15 seconds
        wait_time = random.uniform(5, 15)
        print(f"Ticker: Waiting {wait_time:.2f} seconds befor
e next request...")
        time.sleep(wait_time)

    print("Download task complete.")

# Usage
# Based on your input: 9236 is the last issue of 2025
download_gazette_archive(start_id=9100, end_id=9236)
```

2\. Why this works for your project

- **Direct PDF Access**: By using the `printall-1.php` endpoint, the server does the work of merging the images into a single document for you.
- **User-Agent Mimicry**: The `headers` dictionary makes your script look like a standard Google Chrome browser rather than a Python script.
- **Referer Header**: This tells the server you are coming from their own homepage, which helps pass simple security checks.

- **The Ticker**: Using `random.uniform` ensures your requests don't follow a perfect, robotic rhythm (e.g., exactly every 10 seconds), which is a common red flag for firewalls.

3\. Verification of Content

The PDF files generated by this site use `%PDF-1.4` formatting. If you open a downloaded file and it looks like gibberish in a text editor, that is normal—it's a binary file containing compressed streams and font descriptors for Arabic text.

Would you like me to add a "resume" feature to this script so it can pick up where it left off if your internet cuts out?

you asked

message time: 2026-02-02 12:42:45

okay cool, can we also have the downloads end up in my drive? in a specific folder?

---

gemini response

To move these downloads directly to a specific folder in your Google Drive, we need to integrate the **Google Drive API**. Since you are working in a Python environment, this is the most reliable way to ensure the files aren't just sitting on a local server but are organized in your cloud storage.

1\. Prerequisites

Before running the code, you will need to:

- **Enable the Drive API**: Go to the Google Cloud Console, create a project, and enable the Google Drive API.
- **Create Credentials**: Create an "OAuth 2.0 Client ID" and download the `credentials.json` file to your project folder.
- **Install Libraries**: Run `pip install google-auth-oauthlib google-api-python-client requests`.

2\. The Integrated "Drive-Sync" Scraper

This script combines the **PDF downloader**, the **randomized ticker**, and the **Google Drive Uploader**.

```
import os
import time
import random
import requests
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload

# If modifying these scopes, delete the file token.json.
SCOPES = ['<https://www.googleapis.com/auth/drive.file>']

def get_drive_service():
    creds = None
    if os.path.exists('token.json'):
        creds = Credentials.from_authorized_user_file('token.
json', SCOPES)
    if not creds or not creds.valid:
        flow = InstalledAppFlow.from_client_secrets_file('cre
dentials.json', SCOPES)
        creds = flow.run_local_server(port=0)
        with open('token.json', 'w') as token:
            token.write(creds.to_json())
    return build('drive', 'v3', credentials=creds)

def upload_to_drive(service, file_path, folder_id):
    file_metadata = {
        'name': os.path.basename(file_path),
        'parents': [folder_id]
    }
    media = MediaFileUpload(file_path, mimetype='application/
```

```
pdf')
    file = service.files().create(body=file_metadata, media_b
ody=media, fields='id').execute()
    print(f"Uploaded to Drive. File ID: {file.get('id')}")

def scrape_to_drive(start_id, end_id, drive_folder_id):
    service = get_drive_service()
    base_url = "<https://jo.pcm.gov.lb/pdfs/printall-1.php>"
    headers = {'User-Agent': 'Mozilla/5.0'}

    for issue_id in range(start_id, end_id + 1):
        filename = f"Lebanese_Gazette_{issue_id}.pdf"

        # 1. Download locally first (temporary)
        params = {'issueId': issue_id, 'current': 1}
        response = requests.get(base_url, params=params, head
ers=headers)

        if response.status_code == 200:
            with open(filename, 'wb') as f:
                f.write(response.content)

            # 2. Upload to the specific Drive folder
            upload_to_drive(service, filename, drive_folder_i
d)

            # 3. Clean up local file
            os.remove(filename)
        else:
            print(f"Issue {issue_id} not found.")

        # 4. The Ticker
        time.sleep(random.uniform(5, 12))

# Usage: Replace 'YOUR_FOLDER_ID' with the long string from y
```

```
our Drive folder URL
# scrape_to_drive(9100, 9236, 'YOUR_FOLDER_ID')
```

3\. How to find your `folder_id`

1. Open your Google Drive in a browser.
2. Navigate to the specific folder where you want the Gazettes to go.
3. Look at the URL in your browser address bar. It will look like: `drive.google.com/drive/folders/1A2B3C4D5E6F7G8H9I0J...`
4. The string of random characters after `/folders/` is your **Folder ID**.

4\. Why this is better for your project

- **Direct Cloud Storage**: You won't run out of local disk space, and the files are immediately available for collaboration or backup.
- **Security**: The `token.json` file ensures you only have to log in once; after that, the script runs autonomously.
- **Cleanliness**: The script deletes the PDF from your computer as soon as it is successfully uploaded to Drive, keeping your local workspace tidy.

Would you like me to show you how to add a "Resume" check so the script skips issues already present in your Google Drive folder?

you asked

message time: 2026-02-02 12:44:17

can't i just use colab?

gemini response

Yes, you can absolutely use Google Colab\! In fact, it's actually **easier** because it simplifies the authentication process. Instead of setting up complex API

credentials, you can directly "mount" your Google Drive as a local folder in the Colab environment.

How to use Colab for your Scraper

1. **Mount your Drive**: Click the "Folder" icon on the left sidebar in Colab and select the "Mount Drive" button, or run the code snippet below.
2. **Run the Scraper**: Use the script below. It downloads each issue to a temporary Colab folder and then moves it to your specific Drive folder.

<!-- end list -->

```
import os
import time
import random
import requests
from google.colab import drive

# 1. Mount Google Drive
drive.mount('/content/drive')

# 2. Set your destination (Replace with your actual folder pa
th)
# Tip: Right-click your folder in the Colab sidebar and selec
t "Copy path"
drive_folder = '/content/drive/MyDrive/Lebanese_Gazette_Proje
ct/'
os.makedirs(drive_folder, exist_ok=True)

def colab_scraper(start_id, end_id):
    base_url = "<https://jo.pcm.gov.lb/pdfs/printall-1.php>"
    headers = {'User-Agent': 'Mozilla/5.0'}

    for issue_id in range(start_id, end_id + 1):
        params = {'issueId': issue_id, 'current': 1}
        print(f"Downloading Issue {issue_id}...")
```

```
        try:
            response = requests.get(base_url, params=params,
headers=headers)

            if response.status_code == 200:
                file_path = os.path.join(drive_folder, f"Issu
e_{issue_id}.pdf")
                with open(file_path, 'wb') as f:
                    f.write(response.content)
                print(f"Saved to Drive: {file_path}")
            else:
                print(f"Issue {issue_id} not found (Status {r
esponse.status_code})")

        except Exception as e:
            print(f"Error at {issue_id}: {e}")

        # The Ticker (Anti-suspicion delay)
        time.sleep(random.uniform(5, 12))

# 3. Start the process
colab_scraper(9100, 9236)
```

Why Colab is better for this project:

- **No Setup**: You don't need to install Python or libraries like `requests` on your computer; they are pre-installed in Colab.
- **Cloud-to-Cloud**: The data moves from the Lebanese Gazette server to Google's servers directly. It doesn't use your home internet bandwidth for the upload.
- **Easy Authentication**: You just click a "Connect to Google Drive" prompt instead of managing `credentials.json` files.

A Note on Colab Constraints:

Colab runtimes are temporary. If you are scraping a very large range of issues (e.g., several years), the runtime might time out. Because your files are saved to your Drive instantly, you can just restart the script from the last `issueId` if it stops.

Would you like me to add a line of code that automatically checks your Drive folder and skips any `issueId` that has already been downloaded?

you asked

message time: 2026-02-02 12:49:16

okay so the range is 9236 to 9156

---

gemini response

Since you have the specific range of **9156 to 9236**, we can finalize the script. I've added a "check" feature that looks at your Drive folder before downloading; this way, if the script stops, you can just run it again and it won't waste time (or look suspicious) by redownloading files you already have.

Final Colab Scraper Script

```
import os
import time
import random
import requests
from google.colab import drive

# 1. Mount Google Drive
drive.mount('/content/drive')

# 2. Set your destination folder path
# Change 'My Gazette Project' to your preferred folder name
drive_folder = '/content/drive/MyDrive/My Gazette Project/'
os.makedirs(drive_folder, exist_ok=True)
```

```
def lebanese_gazette_sync(start_id, end_id):
    base_url = "<https://jo.pcm.gov.lb/pdfs/printall-1.php>"
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x
64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 S
afari/537.36'
    }

    # Get a list of IDs already in the folder to avoid duplic
ates
    existing_files = os.listdir(drive_folder)

    # We loop backwards from newest (9236) to oldest (9156)
    for issue_id in range(end_id, start_id - 1, -1):
        filename = f"Issue_{issue_id}.pdf"
        file_path = os.path.join(drive_folder, filename)

        # Check if file exists in Drive
        if filename in existing_files:
            print(f"Skipping {filename} - already exists.")
            continue

        print(f"Requesting Issue {issue_id}...")
        params = {'issueId': issue_id, 'current': 1}

        try:
            response = requests.get(base_url, params=params,
headers=headers, timeout=60)

            if response.status_code == 200 and b'%PDF' in res
ponse.content[:4]:
                with open(file_path, 'wb') as f:
                    f.write(response.content)
                print(f"✅ Successfully saved to Drive: {file
name}")
```

```
            else:
                print(f"❌ Issue {issue_id} not available or
returned invalid data.")

        except Exception as e:
            print(f"⚠ Error at ID {issue_id}: {e}")

        # The Ticker: Random delay to mimic human behavior
        wait = random.uniform(8, 20)
        print(f"Waiting {wait:.1f} seconds...")
        time.sleep(wait)

# 3. Execute for your specific range
lebanese_gazette_sync(9156, 9236)
```

Why this is optimized for your project:

- **Resume Capability**: If Colab disconnects at issue 9200, simply hit "Play" again. The script will see the files for 9236–9200 in your Drive and skip directly to 9199.
- **Validation**: It checks the first few bytes of the file for `%PDF` to ensure the server actually sent a document and not an error page.
- **Human-like Ticker**: I bumped the delay to 8–20 seconds. Since you only have 81 issues to download, this will finish in about 15–20 minutes—slow enough to stay under the radar but fast enough for a coffee break.

Would you like me to help you write a quick block of code to list the titles of the issues once they are all in your Drive?

you asked

message time: 2026-02-02 12:55:06

This only downloads the first image of the pdf, this is a screenshot from the console

gemini response

Based on your screenshot of the console, the website structure has changed how it handles images. The `printall-1.php` endpoint we were using might only be pulling the first page because the site is now lazy-loading individual `.png` files via a JavaScript-heavy viewer.

Specifically, the HTML shows that each page is stored as a separate image in a year-specific directory, such as `http://jo.pcm.gov.lb/egazette_png/2025/9236/1.png`.

To get the **entire** issue, we need to adapt the scraper to find all these image links and then combine them into a single PDF.

Updated Colab Scraper (Multi-Page Version)

This script will visit each `issueId`, find every page image (1.png, 2.png, etc.), and merge them into one PDF per issue in your Drive.

```
import os
import time
import random
import requests
from bs4 import BeautifulSoup
from PIL import Image # For merging images to PDF
from google.colab import drive

# 1. Mount Drive
drive.mount('/content/drive')
drive_folder = '/content/drive/MyDrive/My Gazette Project/'
os.makedirs(drive_folder, exist_ok=True)

def scrape_full_issue(issue_id, year="2025"):
    # The viewer page that lists all images
    url = f"<https://jo.pcm.gov.lb/allimages.php?issueId={iss
ue_id}>"
    headers = {'User-Agent': 'Mozilla/5.0'}
```

```
    try:
        response = requests.get(url, headers=headers)
        soup = BeautifulSoup(response.text, 'html.parser')

        # Find all spans with class 'img-test' as seen in you
r screenshot
        spans = soup.find_all('span', class_='img-test')
        image_urls = [s.get('data-url') for s in spans if s.g
et('data-url')]

        if not image_urls:
            print(f"No pages found for Issue {issue_id}.")
            return

        print(f"Found {len(image_urls)} pages for Issue {issu
e_id}. Downloading...")

        page_images = []
        for img_url in image_urls:
            img_data = requests.get(img_url, stream=True).raw
            img = Image.open(img_data).convert('RGB')
            page_images.append(img)

        # Save all images as a single PDF
        if page_images:
            pdf_path = os.path.join(drive_folder, f"Issue_{is
sue_id}.pdf")
            page_images[0].save(pdf_path, save_all=True, appe
nd_images=page_images[1:])
            print(f"✅ Saved full PDF for Issue {issue_id}")

    except Exception as e:
        print(f"⚠ Failed Issue {issue_id}: {e}")

# 2. Run for your range
for iid in range(9236, 9155, -1):
```

```
    scrape_full_issue(iid)
    time.sleep(random.uniform(5, 10)) # Ticker
```

Why the previous one failed

Looking at your screenshot, the site uses `<span>` tags with a `data-url` attribute to store the image paths. The "Print" function you clicked earlier likely only triggers the "current" page (page 1) because the other pages haven't been loaded into the browser's memory yet.

Key Improvements in this Script:

- **Direct URL Extraction**: It reads the `data-url` directly from the `<span>` tags shown in your console.
- **Image Merging**: It uses the `PIL` (Pillow) library to stitch all those `.png` files into one clean PDF so it’s easy for you to read later.
- **Memory Efficiency**: By converting to 'RGB' and saving as a single file, it keeps the file size manageable in your Drive.

Would you like me to add a progress bar so you can see exactly how many pages are left for each issue?

you asked

message time: 2026-02-02 13:02:34

it doesn't find any pages

---

gemini response

If the previous script isn't finding any pages, it's likely because the website loads that information **dynamically using JavaScript**. Standard libraries like `requests` only see the initial, "empty" HTML shell before the images actually appear.

Based on your screenshot, we can bypass the complex scraping altogether. The image URLs follow a very predictable pattern: `http://jo.pcm.gov.lb/egazette_png/{year}/{issueId}/{page_number}.png`

The "Pattern-Based" Colab Scraper

Instead of trying to find links on the page, this script will simply "guess" the page numbers (1, 2, 3...) until it hits a 404 error, then move to the next issue.

```
import os
import time
import random
import requests
from PIL import Image
from io import BytesIO
from google.colab import drive

# 1. Mount Drive
drive.mount('/content/drive')
drive_folder = '/content/drive/MyDrive/My Gazette Project/'
os.makedirs(drive_folder, exist_ok=True)

def download_issue_by_pattern(issue_id, year="2025"):
    page_images = []
    page_num = 1

    print(f"--- Processing Issue {issue_id} ---")

    while True:
        # Construct the direct image URL based on your screen
shot's pattern
        img_url = f"<http://jo.pcm.gov.lb/egazette_png/{yea
r}/{issue_id}/{page_num}.png>"

        try:
            response = requests.get(img_url, timeout=10)
```

```
            # If we get a 200, the page exists. Keep going.
            if response.status_code == 200:
                img = Image.open(BytesIO(response.content)).c
onvert('RGB')
                page_images.append(img)
                print(f"Loaded page {page_num}")
                page_num += 1
            else:
                # If we get a 404 or other error, we've likel
y hit the end of the issue
                print(f"End of issue reached at page {page_nu
m-1}.")
                break

        except Exception as e:
            print(f"Error at page {page_num}: {e}")
            break

    # Save the gathered images as a PDF
    if page_images:
        pdf_path = os.path.join(drive_folder, f"Issue_{issue_
id}.pdf")
        page_images[0].save(pdf_path, save_all=True, append_i
mages=page_images[1:])
        print(f"✅ Saved full PDF to Drive for Issue {issue_i
d}")
    else:
        print(f"❌ Could not find any pages for Issue {issue_
id}.")

# 2. Run your specific range
for iid in range(9236, 9155, -1):
    download_issue_by_pattern(iid)
    # Ticker: Be gentle with the server
    time.sleep(random.uniform(5, 10))
```

Why this version should work:

- **Ignores JavaScript**: It doesn't care if the website uses lazy-loading or React because it requests the raw image files directly from the server.
- **Pillow Integration**: It uses the `PIL` (Pillow) library to handle the `.png` files and compile them into a PDF.
- **Automatic Termination**: The `while True` loop automatically stops for each issue once it can't find a next page (e.g., trying to find `page 15.png` when the issue only has 14 pages).

Next Step: Would you like me to add a "timeout" check so that if the server is temporarily down, the script waits a minute and tries again instead of skipping the issue?

you asked

message time: 2026-02-02 13:33:01

okay i found this

---

gemini response

Great find\! Your screenshots confirm that the Lebanese Gazette is serving pages as direct image files following a clear pattern:

`http://jo.pcm.gov.lb/egazette_png/{year}/{issueId}/{page_number}.png`

Because these pages are listed as separate `.png` files in the code, the "Print" function fails to find a single PDF. To download the **full issue**, your scraper must check for each page number sequentially (1.png, 2.png, etc.) until it hits a "404 Not Found" error.

Optimized Colab Scraper for your Project

This script uses the `requests` library to check if a page exists without crashing and uses `Pillow` to stitch them into a single PDF directly in your Drive.

```
import os
import time
import random
import requests
from PIL import Image
from io import BytesIO
from google.colab import drive

# 1. Mount Drive
drive.mount('/content/drive')
drive_folder = '/content/drive/MyDrive/Lebanese_Gazette_202
5/'
os.makedirs(drive_folder, exist_ok=True)

def download_complete_gazette(issue_id, year="2025"):
    pages = []
    current_page = 1

    print(f"\\n--- Starting Issue {issue_id} ---")

    while True:
        img_url = f"<http://jo.pcm.gov.lb/egazette_png/{yea
r}/{issue_id}/{current_page}.png>"

        try:
            # Check if the page exists
            response = requests.get(img_url, timeout=10)

            if response.status_code == 200:
                # Load image into memory
                img = Image.open(BytesIO(response.content)).c
onvert('RGB')
                pages.append(img)
                print(f"Loaded page {current_page}")
                current_page += 1
```

```
            else:
                # 404 means we've reached the end of the issu
e
                print(f"Finished: Issue {issue_id} has {curre
nt_page-1} pages.")
                break

        except Exception as e:
            print(f"Connection error at page {current_page}:
{e}")
            break

    # Save as a single PDF if pages were found
    if pages:
        pdf_name = f"Gazette_Issue_{issue_id}.pdf"
        save_path = os.path.join(drive_folder, pdf_name)
        pages[0].save(save_path, save_all=True, append_images
=pages[1:])
        print(f"✅ Saved to Drive: {pdf_name}")
    else:
        print(f"❌ Error: No pages found for Issue {issue_i
d}.")

# 2. Run for your range (9236 down to 9156)
for iid in range(9236, 9155, -1):
    download_complete_gazette(iid)
    # Ticker: Wait between issues to avoid being flagged
    time.sleep(random.uniform(5, 10))
```

Why this works for you:

- **Pattern Recognition**: Your screenshot of issue `9233` shows it goes up to at least page `248`. This script will find every single one of them automatically by incrementing `current_page`.

- **Reliability**: It uses `response.status_code == 200` to verify each image exists before trying to process it, preventing "File Not Found" errors.
- **Resource Optimization**: It only downloads headers if needed and compiles the PDF in a way that saves space in your Google Drive.

Download Images from URL using Python
This video demonstrates the core logic of using the requests library to verify and download image files efficiently.

Would you like me to add a "Resume" feature that checks which Issue IDs are already in your Drive folder so you don't redownload them?

you asked

message time: 2026-02-02 13:35:06

it stops at 8 pages on the first one, and 4 on the second one even though there are over a hundred pages

---

gemini response

It sounds like the server is doing one of two things: either it is returning a **"blank" image** or a placeholder page (still with a `200 OK` status) once it suspects a scraper, or it is **rate-limiting** you by cutting off the connection after a few rapid-fire requests.

Since the site isn't returning a `404` to tell the script to stop, the `while True` loop thinks there are infinite pages but is just receiving "empty" data.

The "Smart" Multi-Page Scraper

We need to update the script to **validate the image size**. If the server sends a tiny placeholder (like a 0-byte file or a 1×1 pixel image), the script should recognize that as the "real" end of the issue.

```
import os
import time
```

```
import random
import requests
from PIL import Image
from io import BytesIO
from google.colab import drive

# 1. Mount Drive
drive.mount('/content/drive')
drive_folder = '/content/drive/MyDrive/My Gazette Project/'
os.makedirs(drive_folder, exist_ok=True)

def download_fix(issue_id, year="2025"):
    pages = []
    current_page = 1
    # Increase the delay to avoid being flagged by the serve
r's "bouncer"

    print(f"\\n--- Scraping Issue {issue_id} ---")

    while True:
        img_url = f"<http://jo.pcm.gov.lb/egazette_png/{yea
r}/{issue_id}/{current_page}.png>"

        try:
            # Use a realistic User-Agent to look like a perso
n browsing
            headers = {'User-Agent': 'Mozilla/5.0 (Macintosh;
Intel Mac OS X 10_15_7) AppleWebKit/537.36'}
            response = requests.get(img_url, headers=headers,
timeout=15)

            # Check for success AND verify it's not a tiny em
pty file
            if response.status_code == 200 and len(response.c
ontent) > 5000:
                img = Image.open(BytesIO(response.content)).c
```

```
onvert('RGB')
                pages.append(img)
                print(f"Page {current_page} ok ({len(respons
e.content)} bytes)")
                current_page += 1

                # Critical: Add a small delay between pages s
o we don't look like a bot
                time.sleep(random.uniform(1.5, 3.0))
            else:
                print(f"Reached end or empty page at {current
_page}.")
                break

        except Exception as e:
            print(f"Stop at page {current_page}: {e}")
            break

    if pages:
        pdf_path = os.path.join(drive_folder, f"Issue_{issue_
id}.pdf")
        pages[0].save(pdf_path, save_all=True, append_images=
pages[1:])
        print(f"✅ Created PDF: Issue {issue_id}")
    else:
        print(f"❌ Failed to find valid pages for {issue_i
d}.")

# 2. Process your range
for iid in range(9236, 9155, -1):
    download_fix(iid)
    # Long wait between full issues
    print("Taking a breather...")
    time.sleep(random.uniform(10, 20))
```

Why this addresses the 8-page cutoff:

- **Size Validation**: By checking `len(response.content) > 5000`, the script ignores small error images or empty pixels that the server might be sending to "trick" your scraper.
- **Intra-Page Ticker**: I added a `time.sleep` **inside** the page loop. If you request 100 pages in 1 second, the server will block you after page 8. If you request them every 2 seconds, you look like a fast human reader.
- **Connection Stability**: The `timeout=15` ensures that if the server hangs while trying to throttle you, the script doesn't just sit there forever.

Web Scraping with Python - Image Downloading Tutorial
This video is a great resource for understanding how to handle and save bulk image data using Python's `requests` library.

Would you like me to add a "Retry" block? If the server fails once, the script can wait 60 seconds and try that page again before giving up.

you asked

message time: 2026-02-02 13:52:47

okay so when i click inspect, i can see all the links to the png's, so we need to find a way to download them either from the console directly, and then put them together into a pdf or do it some other way

---

gemini response

Since you can see all the `.png` links in the console, we can use a two-step "Power User" approach. We will use a small JavaScript snippet in your browser console to extract all the links at once, and then use a Python script in Colab to download and stitch them into a PDF.

Step 1: Extract all links from the Console

Open the page for a specific issue (e.g., Issue 9236), open your **Inspect Element** console (F12 or Cmd+Opt+I), and paste this code. It will find every link and copy them to your clipboard automatically:

```
// Find all the image links from the span tags you found
let images = document.querySelectorAll('span.img-test');
let links = Array.from(images).map(span => span.getAttribute
('data-url')).filter(url => url);

// Copy them to your clipboard as a list
copy(links.join('\\n'));
print("Copied " + links.length + " links to your clipboar
d!");
```

Step 2: Download and Merge in Colab

Now that you have the links copied, go to your Colab notebook. This script will take that list of links, download the images, and merge them into one PDF in your Drive.

```
import os
import requests
from PIL import Image
from io import BytesIO
from google.colab import drive

# 1. Mount Drive
drive.mount('/content/drive')
drive_folder = '/content/drive/MyDrive/My Gazette Project/'
os.makedirs(drive_folder, exist_ok=True)

# 2. Paste your links between the triple quotes below
links_text = """
PASTE_YOUR_COPIED_LINKS_HERE
"""
```

```
def process_issue_from_links(links_str, issue_name):
    urls = [url.strip() for url in links_str.strip().split
('\\n') if url.strip()]
    if not urls:
        print("No links found. Please paste the links into th
e script.")
        return

    pages = []
    print(f"Starting download for {issue_name}...")

    headers = {'User-Agent': 'Mozilla/5.0'}

    for i, url in enumerate(urls):
        try:
            response = requests.get(url, headers=headers, tim
eout=15)
            if response.status_code == 200:
                img = Image.open(BytesIO(response.content)).c
onvert('RGB')
                pages.append(img)
                if (i+1) % 10 == 0: print(f"Downloaded {i+1}
pages...")
        except Exception as e:
            print(f"Failed at page {i+1}: {e}")

    if pages:
        pdf_path = os.path.join(drive_folder, f"{issue_name}.
pdf")
        pages[0].save(pdf_path, save_all=True, append_images=
pages[1:])
        print(f"✅ Created PDF: {pdf_path}")
```

```
# 3. Run it
process_issue_from_links(links_text, "Issue_9236")
```

Why this bypasses the "8-page" limit

The reason the previous script stopped early is that the website likely uses **Lazy Loading**. The images only exist in the "Inspect Element" view once you've scrolled through the page.

- **The Console Trick**: By using the JavaScript snippet while you are on the page, you are grabbing exactly what your browser sees.
- **Manual Control**: Since you are giving the Colab script the exact list of URLs, it doesn't have to "guess" anymore; it just goes through the list one by one.
- **Pillow (PIL)**: This library is the industry standard for taking a list of image objects and saving them as a multi-page PDF.

How to Convert Multiple Images to a Single PDF using Python
This video provides a clear walkthrough on how to use the Pillow library to combine multiple images into a single, high-quality PDF document.

Would you like me to show you how to automate the "scrolling" part in the browser so you don't have to scroll down to reveal all 200+ images manually?

you asked

message time: 2026-02-02 14:17:30

import os
import time
import random
import requests
from PIL import Image
from io import BytesIO
from google.colab import drive

1. Mount Drive

drive.mount('/content/drive')
drive_folder = '/content/drive/MyDrive/Lebanese_Gazette_2025/'
os.makedirs(drive_folder, exist_ok=True)

def download_complete_gazette(issue_id, year="2025"):
pages = []
current_page = 1

```
print(f"\\n--- Starting Issue {issue_id} ---")

while True:
    img_url = f"<http://jo.pcm.gov.lb/egazette_png/{year}/{issue_id}/{current_page}.png>"

    try:
        # Check if the page exists
        response = requests.get(img_url, timeout=10)

        if response.status_code == 200:
            # Load image into memory
            img = Image.open(BytesIO(response.content)).convert('RGB')
            pages.append(img)
            print(f"Loaded page {current_page}")
            current_page += 1
        else:
            # 404 means we've reached the end of the issue
            print(f"Finished: Issue {issue_id} has {current_page-1} pages.")
            break

    except Exception as e:
        print(f"Connection error at page {current_page}: {e}")
        break

# Save as a single PDF if pages were found
if pages:
    pdf_name = f"Gazette_Issue_{issue_id}.pdf"
    save_path = os.path.join(drive_folder, pdf_name)
    pages[0].save(save_path, save_all=True, append_images=pages[1:])
    print(f"✅ Saved to Drive: {pdf_name}")
else:
    print(f"❌ Error: No pages found for Issue {issue_id}.")
```

2. Run for your range (9236 down to 9156)

for iid in range(9236, 9155, -1):
download_complete_gazette(iid)
# Ticker: Wait between issues to avoid being flagged
time.sleep(random.uniform(5, 10)) can you format this for vscode please and i want the output folder to be called 'La Gazette'

---

gemini response

To move this from Google Colab to **VS Code**, we need to remove the Colab-specific `drive.mount` and replace it with standard local file handling. I have also updated the folder name to **'La Gazette'** as requested.

VS Code Preparation

1. **Open VS Code** and create a new folder for your project.
2. **Create a file** named `scraper.py` and paste the code below.
3. **Install the required library**: Open your terminal in VS Code (Ctrl+`) and run:
   pip install requests Pillow\`

The Updated Script

```
import os
import time
import random
import requests
from PIL import Image
from io import BytesIO

# 1. Setup local output folder
# This will create a 'La Gazette' folder in the same director
y as your script
output_folder = 'La Gazette'
if not os.path.exists(output_folder):
    os.makedirs(output_folder)
    print(f"Created folder: {output_folder}")
```

```
def download_complete_gazette(issue_id, year="2025"):
    pages = []
    current_page = 1

    print(f"\\n--- Starting Issue {issue_id} ---")

    # Using a session for better performance and to look more
like a browser
    session = requests.Session()
    session.headers.update({
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x
64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 S
afari/537.36'
    })

    while True:
        img_url = f"<http://jo.pcm.gov.lb/egazette_png/{yea
r}/{issue_id}/{current_page}.png>"

        try:
            response = session.get(img_url, timeout=15)

            # Check if the page exists and has actual content
(greater than 5KB)
            if response.status_code == 200 and len(response.c
ontent) > 5000:
                img = Image.open(BytesIO(response.content)).c
onvert('RGB')
                pages.append(img)
                print(f"Loaded page {current_page} ({len(resp
onse.content)} bytes)")
                current_page += 1

                # Small delay between pages to prevent the se
rver from cutting you off
```

```
                time.sleep(random.uniform(0.5, 1.5))
            else:
                print(f"Reached end of issue at page {current
_page - 1}.")
                break

        except Exception as e:
            print(f"Connection error at page {current_page}:
{e}")
            break

    # Save as a single PDF if pages were found
    if pages:
        pdf_name = f"Gazette_Issue_{issue_id}.pdf"
        save_path = os.path.join(output_folder, pdf_name)

        # Save pages as a PDF
        pages[0].save(save_path, save_all=True, append_images
=pages[1:])
        print(f"✅ Successfully saved: {save_path}")
    else:
        print(f"❌ Error: No valid pages found for Issue {iss
ue_id}.")

# 2. Main execution loop (9236 down to 9156)
if __name__ == "__main__":
    try:
        for iid in range(9236, 9155, -1):
            download_complete_gazette(iid)

            # Ticker: Wait between full issues to avoid suspi
cion
            wait_time = random.uniform(10, 20)
            print(f"Waiting {wait_time:.1f} seconds before th
e next issue...")
            time.sleep(wait_time)
```

```
    except KeyboardInterrupt:
        print("\\nStopping script manually...")
```

Key Changes for VS Code:

- **Local Storage**: Removed `google.colab` and `drive.mount`. The script now creates the **'La Gazette'** folder on your hard drive.
- **Session Handling**: Added `requests.Session()` which is more efficient for making multiple requests to the same server.
- **Anti-Throttling Fix**: Based on your previous issue where it stopped at page 8, I added `len(response.content) > 5000` to ensure the script doesn't treat empty/broken images as valid pages, and added a sub-delay **between pages** to stay under the radar.
- **Main Guard**: Added `if __name__ == "__main__":`, which is standard practice for Python scripts in IDEs like VS Code.

Would you like me to add a line that converts these local PDFs into a searchable text format (OCR) once they are downloaded?

you asked

message time: 2026-02-02 14:19:54

louaimroueh@Michael-Jackson La Gazette % pip install requests Pillow
zsh: command not found: pip
louaimroueh@Michael-Jackson La Gazette %

gemini response

The error `zsh: command not found: pip` occurs because macOS typically links the `pip` command to Python 2 (which is no longer included), while your current

environment uses **Python 3**. On modern Macs, the package manager is usually named `pip3`.

The Quick Fix

Try running the command using `pip3` instead of `pip`. In your VS Code terminal, type:

```
pip3 install requests Pillow
```

The "Universal" Fix (Recommended)

If `pip3` also fails, you can bypass the `PATH` entirely by calling the Python module directly. This is the most reliable method for macOS:

```
python3 -m pip install requests Pillow
```

If it still says "command not found"

If neither works, it means `pip` isn't installed for your Python version yet. You can "bootstrap" it using this built-in command:

```
python3 -m ensurepip --upgrade
```

After that, try the `python3 -m pip install` command again.

Pro-Tip for your Project

Since you are working on a specific project, it's a "best practice" to avoid installing packages globally. You can create a **Virtual Environment** inside your 'La Gazette' folder. This keeps your project isolated and clean:

1. **Create the environment**: `python3 -m venv venv`
2. **Activate it**: `source venv/bin/activate`
3. **Install your tools**: `pip install requests Pillow` (Inside the environment, just `pip` will work\!)

Would you like me to help you set up the alias so that `pip` always works for you in the future?

Fixing 'command not found: pip' on Mac
This video provides a visual walkthrough of the different ways to resolve the `pip` command error on macOS and how to correctly install Python packages.

you asked

message time: 2026-02-02 14:22:09

okay i did that, what happens next?

---

gemini response

Now that you've successfully installed **requests** and **Pillow**, you are ready to run the scraper and start populating your 'La Gazette' folder.

1\. Running the Script

1. **Open the file**: In VS Code, make sure your `scraper.py` file is open.
2. **Execute**: You can run it by clicking the **"Play" button** in the top-right corner of VS Code, or by typing this in your terminal:

```
python3 scraper.py
```

2\. What to Expect

As soon as the script starts, watch your terminal and your sidebar:

- **The Folder**: A new folder named **La Gazette** will appear in your project directory.
- **The Progress**: You will see the terminal print "Starting Issue 9236..." followed by "Loaded page 1", "Loaded page 2", etc.

- **The Files**: Once an issue finishes (e.g., reaches the end of the pages), a file named `Gazette_Issue_9236.pdf` will appear inside the 'La Gazette' folder.

3\. Troubleshooting Common Next Steps

- **Script stops early**: If it stops after a few pages again, try increasing the `time.sleep` values in the script. Some government servers are very sensitive to rapid requests.
- **Permission Error**: If it says "Permission Denied" when creating the folder, make sure you aren't trying to save to a protected system directory.
- **Stopping the script**: If you need to stop it for any reason, just click in the terminal and press `Ctrl + C`.

4\. Verification

Once the first PDF appears, open it to ensure all pages are there and the Arabic text is clear. Since we are stitching PNGs into a PDF, the quality should match exactly what you saw on the website.

Would you like me to add a "Status Report" feature to the script that creates a text file summarizing which issues were successfully downloaded and which ones failed?

you asked

message time: 2026-02-02 14:28:37

```
louaimroueh@Michael-Jackson La Gazette % /opt/homebrew/bin/p
ython3 "/Users/louaimroueh/Desktop/La Gazette/scraper.py"
^C
louaimroueh@Michael-Jackson La Gazette % /opt/homebrew/bin/p
ython3 "/Users/louaimroueh/Desktop/La Gazette/scraper.py"
Traceback (most recent call last):
  File "/Users/louaimroueh/Desktop/La Gazette/scraper.py", line 4, in <module>
    import requests
ModuleNotFoundError: No module named 'requests'
louaimroueh@Michael-Jackson La Gazette %
```

gemini response

The `ModuleNotFoundError` is happening because the terminal is using the global Homebrew version of Python (`/opt/homebrew/bin/python3`), but the **requests** and **Pillow** libraries were likely installed in a different location or a virtual environment.

To fix this, you need to tell that specific version of Python to install the missing libraries.

The Fix

Run this command in your terminal. It uses the exact same path the script is trying to use:

```
/opt/homebrew/bin/python3 -m pip install requests Pillow
```

Why this happened

On macOS, you can have multiple "versions" of Python installed (the Apple default, the Homebrew version, and others). If you installed `requests` using just `pip3`, it might have gone to a different version's folder. By using the full path followed by `-m pip`, you guarantee the libraries are installed exactly where your script is looking for them.

What to do next

1. Run the install command above.
2. Once it finishes, run your script again:

```
/opt/homebrew/bin/python3 "/Users/louaimroueh/Desktop/La Gazette/scraper.py"
```

Pro-Tip for VS Code

In the bottom-right corner of your VS Code window, you should see something like **"Python 3.x.x"**. Click on it and select the interpreter that matches `/opt/homebrew/bin/python3`. This will sync the VS Code terminal and the editor so you don't run into this "Module Not Found" error again.

Once the first PDF successfully downloads, would you like me to show you how to add a "Headless Mode" so the script runs faster in the background?